

SQL Joins

Work Sample by Misti Burton

Scenario

This scenario is based on a fictional company as part of Coursera's Google Cybersecurity Professional Certificate.

You are tasked with analyzing sales and employee data for a small bookstore business. All of the data is to be accessed using MySQL Workbench.

My tasks:

Utilize MySQL to perform the following command types:

1. *JOINS (inner, left, right, self, and cross)*
2. *UNION and UNION ALL*

SQL Queries

Inner Join:

An Inner Join was used to retrieve only the matching records between the sales table and the employee data table where a valid employee ID exists in both datasets. This ensures that only transactions associated with active employees are included in the analysis.

Command:

```
SELECT sales.sale_id, sales.date, employees.employee_name, sales.amount FROM sales  
INNER JOIN employees ON sales.employee_id = employees.employee_id;
```

Left Join:

A Left Join was used to retrieve all sales records along with any matching employee data. This ensures that all sales transactions remain included in the analysis, even if an associated employee record is missing.

Command:

```
SELECT sales.sale_id, sales.date, employees.employee_name, sales.amount FROM sales  
LEFT JOIN employees ON sales.employee_id = employees.employee_id;
```

Right Join:

A Right Join was used to retrieve all employee records along with any matching sales data. This ensures that all employees remain included in the analysis, even if an employee has not processed any sales transactions.

Command:

```
SELECT sales.sale_id, sales.date, employees.employee_name, sales.amount FROM sales  
RIGHT JOIN employees ON sales.employee_id = employees.employee_id;
```

Self-Join:

A Self-Join was used to join the employee table to itself in order to compare records within the same dataset. This allowed us to examine hierarchical relationships between employees, such as matching staff members with their respective managers.

Command:

```
SELECT e1.employee_name AS employee, e2.employee_name AS manager FROM  
employees AS e1 JOIN employees AS e2 ON e1.manager_id = e2.employee_id;
```

Cross Join:

A Cross Join was used to return the Cartesian product of the sales and employee tables. This combined every row from the sales table with every row from the employee table to evaluate all possible combinations.

Command:

```
SELECT sales.sale_id, sales.date, employees.employee_name, sales.amount FROM sales  
CROSS JOIN employees;
```

Union:

A Union operation was used to combine the result sets of two SELECT statements into a single result set while automatically removing duplicate rows. This allowed us to consolidate employee names across multiple regional databases into a single, distinct list.

Command:

```
SELECT employee_name FROM employees_region1 UNION SELECT employee_name FROM  
employees_region2;
```